$SiER^2NA$

Simple Extensible Register-Register Neumann Architecture

Instruction Set Architecture

1.0

October 12, 2026

Contents

1 Preface **2**

2 Glossary **2**

3 Design **3**
- 3.1 Instruction types . . . 3
  - 3.1.1 Register type (R-type) . . . 3
  - 3.1.2 Immediate type (I-type) . . . 3
  - 3.1.3 Immediate jump type (J-type) . . . 3
- 3.2 Instruction encoding . . . 3
  - 3.2.1 R-type . . . 3
  - 3.2.2 I-type . . . 3
  - 3.2.3 J-type . . . 3

4 Core set (CORE32) **3**
- 4.1 Miscellaneous . . . 3
  - 4.1.1 nop - no operation . . . 3
  - 4.1.2 cpuid - CPU identification . . . 3
- 4.2 Load/Store operations . . . 4
  - 4.2.1 ldw - load word . . . 4
  - 4.2.2 ldhz - load half-word zero-extended . . . 4
  - 4.2.3 ldhs - load half-word sign-extended . . . 4
  - 4.2.4 ldbz - load byte zero-extended . . . 4
  - 4.2.5 ldbs - load byte sign-extended . . . 5
  - 4.2.6 lvi - load sign-extended value immediate . . . 5
  - 4.2.7 lli - load lower half-word immediate . . . 5
  - 4.2.8 lui - load upper half-word immediate . . . 5
  - 4.2.9 stw - store word . . . 6
  - 4.2.10 sth - store half-word . . . 6
  - 4.2.11 stb - store byte . . . 6
- 4.3 Register operations . . . 6
  - 4.3.1 mov - move . . . 6
- 4.4 Control flow . . . 7
  - 4.4.1 ja - jump absolute . . . 7

4.4.2 jae - jump absolute if equal . . . 7
4.4.3 jane - jump absolute if not equal . . . 7
4.4.4 jalt - jump absolute if less than . . . 7
4.4.5 jage - jump absolute if greater or equal . . . 8
4.4.6 jr - jump relative . . . 8
4.4.7 jre - jump relative if equal . . . 8
4.4.8 jrne - jump relative if not equal . . . 9
4.4.9 jrlt - jump relative if less than . . . 9
4.4.10 jrge - jump relative if greater or equal . . . 9
4.4.11 calla - call procedure absolute . . . 10
4.4.12 callr - call procedure relative immediate . . . 10
4.4.13 ret - return from procedure . . . 10
4.5 Mathematical operations . . . 10
4.5.1 cmp - compare values . . . 10
4.5.2 add - integer addition . . . 11
4.5.3 sub - integer subtraction . . . 11
4.5.4 addi - integer addition sign-extended immediate . . . 12
4.5.5 mul - integer multiplication . . . 12
4.5.6 div - integer division . . . 12
4.6 Bitwise operations . . . 12
4.6.1 and - bitwise AND . . . 12
4.6.2 or - bitwise OR . . . 13
4.6.3 xor - bitwise XOR . . . 13
4.6.4 shl - shift left . . . 13
4.6.5 shr - shift right . . . 13
4.6.6 shsr - arithmetic (signed) shift right . . . 14
4.6.7 shli - logical shift left immediate . . . 14
4.6.8 shri - logical shift right immediate . . . 14
4.6.9 shri - arithmetic (signed) shift right immediate . . . 14

1 Preface

2 Glossary

- sext18(x) - sign-extend x from bit 17 to 32 bits
- sext16(x) - sign-extend x from bit 15 to 32 bits
- zext16(x) - zero-extend x from bit 15 to 32 bits
- sext8(x) - sign-extend x from bit 7 to 32 bits
- zext8(x) - zero-extend x from bit 7 to 32 bits
- read8(a) - read 8 bits from address a
- read16(a) - read 16 bits from address a
- write8(a, x) - write 8 bit value x to address a
- write16(a, x) - write 16 bit value x to address a

3 Design

3.1 Instruction types

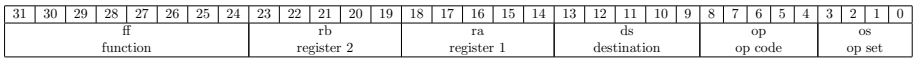
3.1.1 Register type (R-type)

3.1.2 Immediate type (I-type)

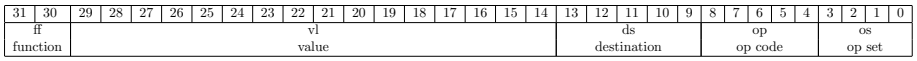
3.1.3 Immediate jump type (J-type)

3.2 Instruction encoding

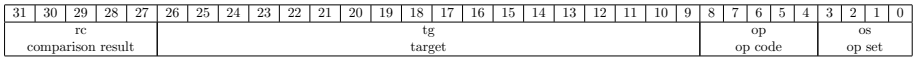
3.2.1 R-type



3.2.2 I-type



3.2.3 J-type



4 Core set (CORE32)

4.1 Miscellaneous

4.1.1 nop - no operation

os	op	ds	ra	rb	ff
0x00	0x00	–	–	–	–

Performs no operation. The number of cycles consumed by this instruction is implementation-dependent and not guaranteed.

4.1.2 cpuid - CPU identification

os	op	ds	ra	rb	ff
0x00	0x01	result	–	–	–

Writes the CPUID structure into *result*. See CPU: CPUID

Implementation

```
1 // implementation-specific
```

4.2 Load/Store operations

4.2.1 ldw - load word

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source	–	0x00

Loads a single word from memory at *source* into *destination*

Implementation

```
1 value = *source
```

4.2.2 ldhz - load half-word zero-extended

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source	–	0x01

Loads a zero-extended half-word from memory at *source* into *destination*

Implementation

```
1 value = zext16(read16(*source))
```

4.2.3 ldhs - load half-word sign-extended

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source	–	0x02

Loads a sign-extended half-word from memory at *source* into *destination*

Implementation

```
1 value = sext16(read16(*source))
```

4.2.4 ldbz - load byte zero-extended

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source	–	0x03

Loads a zero-extended byte from memory at *source* into *destination*

Implementation

```
1 value = zext8(read8(*source))
```

4.2.5 ldbs - load byte sign-extended

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source	–	0x04

Loads a sign-extended byte from memory at *source* into *destination*

Implementation

```
1 value = sext8(read8(*source))
```

4.2.6 lvi - load sign-extended value immediate

os	op	ds	vl	ff
0x00	0x03	destination	value	–

Loads *value* into the lower half-word of *destination*, sign-extending it to the upper-half

Implementation

```
1 destination = sext16(value)
```

4.2.7 lli - load lower half-word immediate

os	op	ds	vl	ff
0x00	0x04	destination	value	0x01

Loads *value* into the lower half-word of *destination*

Implementation

```
1 destination = (*destination & 0xFFFF0000) | value
```

4.2.8 lui - load upper half-word immediate

os	op	ds	vl	ff
0x00	0x04	destination	value	0x02

Loads *value* into the upper half-word of *destination*

Implementation

```
1 destination = (*destination & 0x0000FFFF) | (value <<
    16)
```

4.2.9 stw - store word

os	op	ds	ra	rb	ff
0x00	0x05	&destination	source	–	0x00

Writes a single word into memory at *destination* from *source*

Implementation

```
1 *destination = source
```

4.2.10 sth - store half-word

os	op	ds	ra	rb	ff
0x00	0x05	&destination	source	–	0x01

Writes a half-word into memory at *destination* from *source*

Implementation

```
1 *destination = source >> 16
```

4.2.11 stb - store byte

os	op	ds	ra	rb	ff
0x00	0x05	&destination	source	–	0x02

Writes a byte into memory at *destination* from *source*

Implementation

```
1 *destination = source >> 24
```

4.3 Register operations

4.3.1 mov - move

os	op	ds	ra	rb	ff
0x00	0x06	destination	source	–	–

Copies the value of *source* into *destination*

Implementation

```
1 destination = source
```

4.4 Control flow

4.4.1 ja - jump absolute

os	op	ds	ra	rb	ff
0x00	0x07	&target	–	–	–

Sets the *instruction pointer* to the value of *target*

Implementation

```
1 rip = target
```

4.4.2 jae - jump absolute if equal

os	op	ds	ra	rb	ff
0x00	0x08	&target	cmp	–	0x00

Sets the *instruction pointer* to the value of *target* if the comparison result *cmp* is **EQUAL**

Implementation

```
1 if (cmp & 0b0001) == 1 {
2     rip = target
3 }
```

4.4.3 jane - jump absolute if not equal

os	op	ds	ra	rb	ff
0x00	0x08	&target	cmp	–	0x01

Sets the *instruction pointer* to the value of *target* if the comparison result *cmp* is **NOT EQUAL**

Implementation

```
1 if (cmp & 0b0010) == 1 {
2     rip = target
3 }
```

4.4.4 jalt - jump absolute if less than

os	op	ds	ra	rb	ff
0x00	0x08	&target	cmp	–	0x02

Sets the *instruction pointer* to the value of *target* if the comparison result *cmp* is **LESS THAN**

Implementation

```
1 if (cmp & 0b0100) == 1 {
2     rip = target
3 }
```

4.4.5 jage - jump absolute if greater or equal

os	op	ds	ra	rb	ff
0x00	0x08	&target	cmp	–	0x03

Sets the *instruction pointer* to the value of *target* if the comparison result *cmp* is **GREATER THAN** or **EQUAL**

Implementation

```
1 if (cmp & 0b1000) == 1 || (cmp & 0b0001) == 1 {
2     rip = target
3 }
```

4.4.6 jr - jump relative

os	op	tg	rc
0x00	0x09	target	–

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer*

Implementation

```
1 rip = rip + sext18(target)
```

4.4.7 jre - jump relative if equal

os	op	tg	rc
0x00	0x0A	target	cmp

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer* if the comparison result *cmp* is **EQUAL**

Implementation

```
1 if (cmp & 0b0001) == 1 {
2     rip = rip + sext18(target)
3 }
```

4.4.8 jrne - jump relative if not equal

os	op	tg	rc
0x00	0x0B	target	cmp

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer* if the comparison result *cmp* is **NOT EQUAL**

Implementation

```
1 if (cmp & 0b0010) == 1 {
2     rip = rip + sext18(target)
3 }
```

4.4.9 jrlt - jump relative if less than

os	op	tg	rc
0x00	0x0C	target	cmp

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer* if the comparison result *cmp* is **LESS THAN**

Implementation

```
1 if (cmp & 0b0100) == 1 {
2     rip = rip + sext18(target)
3 }
```

4.4.10 jrge - jump relative if greater or equal

os	op	tg	rc
0x00	0x0D	target	cmp

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer* if the comparison result *cmp* is **GREATER THAN** or **EQUAL**

Implementation

```
1 if (cmp & 0b1000) == 1 || (cmp & 0b0001) == 1 {
2     rip = rip + sext18(target)
3 }
```

4.4.11 calla - call procedure absolute

os	op	ds	ra	rb	ff
0x00	0x0E	&target	–	–	–

Sets the *procedure tail pointer* to the value of the *instruction pointer* and then sets the *instruction pointer* to the value of *target*

Implementation

```
1 rtp = rip + 4
2 rip = target
```

4.4.12 callr - call procedure relative immediate

os	op	tg	rc
0x00	0x0F	&target	–

Sets the *procedure tail pointer* to the value of the *instruction pointer* and then sets the *instruction pointer* to the value of sign-extended *target* relative to the current value of the *instruction pointer*

Implementation

```
1 rtp = rip + 4
2 rip = rip + sext18(target)
```

4.4.13 ret - return from procedure

os	op	ds	ra	rb	ff
0x00	0x10	–	–	–	–

Sets the *instruction pointer* to the value of the *procedure tail pointer*

Implementation

```
1 rip = rtp
```

4.5 Mathematical operations

4.5.1 cmp - compare values

os	op	ds	ra	rb	ff
0x00	0x11	result	a	b	–

Compares the values of *a* and *b* and stores the comparison result in *result*. The comparison result is a 4-bit flag value and maps to the following results:

bit	mnemonic	description
0	EQUAL	if set, a and b are equal
1	NOT EQUAL	if set, a and b are not equal
2	LESS THAN	if set, a is less than b
3	GREATER THAN	if set, a is greater than b

Implementation

```
result = 0

if a == b {
    result = result | (1 << 0)
} else {
    result = result | (1 << 1)

    if a < b {
        result = result | (1 << 2)
    }

    if a > b {
        result = result | (1 << 3)
    }
}
```

4.5.2 add - integer addition

os	op	ds	ra	rb	ff
0x00	0x12	result	a	b	0x00

Adds the values of a and b together and stores the result in *result*

Implementation

```
result = a + b
```

4.5.3 sub - integer subtraction

os	op	ds	ra	rb	ff
0x00	0x12	result	a	b	0x01

Subtracts the value of b from the value of a and stores the result in *result*

Implementation

```
result = a - b
```

4.5.4 addi - integer addition sign-extended immediate

os	op	ds	vl	ff
0x00	0x13	a	b	–

Adds the value of sign-extended *b* and the value of *a* together, storing the result in *a*

Implementation

```
1 a = a + sext16(b)
```

4.5.5 mul - integer multiplication

os	op	ds	ra	rb	ff
0x00	0x14	result	a	b	0x00

Multiplies the values of *a* and *b* together and stores the result in *result*

Implementation

```
1 result = a * b
```

4.5.6 div - integer division

os	op	ds	ra	rb	ff
0x00	0x14	result	a	b	0x01

Divides the value of *a* by the value of *b* stores the result in *result*

- Fractional results are truncated towards 0
- Division by 0 raises **#DBZ**

Implementation

```
1 result = a // b
```

4.5.7 mod - integer remainder

os	op	ds	ra	rb	ff
0x00	0x14	result	a	b	0x02

Stores the division remainder of dividing *a* by *b* in *result*

- Fractional results are truncated towards 0

- Modulo by 0 throws a DIVISION BY ZERO exception

Implementation

```
1 result = a % b
```

4.6 Bitwise operations

4.6.1 and - bitwise AND

os	op	ds	ra	rb	ff
0x00	0x15	result	a	b	0x00

Performs a bitwise AND operation on the values of *a* and *b* and stores the result in *result*

Implementation

```
1 result = a & b
```

4.6.2 or - bitwise OR

os	op	ds	ra	rb	ff
0x00	0x15	result	a	b	0x01

Performs a bitwise OR operation on the values of *a* and *b* and stores the result in *result*

Implementation

```
1 result = a | b
```

4.6.3 xor - bitwise XOR

os	op	ds	ra	rb	ff
0x00	0x15	result	a	b	0x02

Performs a bitwise XOR operation on the values of *a* and *b* and stores the result in *result*

Implementation

```
1 result = a ^ b
```

4.6.4 shl - logical shift left

os	op	ds	ra	rb	ff
0x00	0x16	result	a	n	0x00

Shifts the value of *a* left by *n* places and stores the result in *result*

Implementation

```
1 result = a << n
```

4.6.5 shr - logical shift right

os	op	ds	ra	rb	ff
0x00	0x16	result	a	n	0x01

Shifts the value of *a* right by *n* places and stores the result in *result*

Implementation

```
1 result = a >> n
```

4.6.6 shsr - arithmetic (signed) shift right

os	op	ds	ra	rb	ff
0x00	0x16	result	a	n	0x02

Shifts the value of *a* right by *n* places and stores the result in *result*, sign-replicating it

Implementation

```
1 result = (a >> n) | (a >> n >> (31 - n) << 31)
```

4.6.7 shli - logical shift left immediate

os	op	ds	vl	ff
0x00	0x17	a	n	0x00

Shifts the value of *a* in-place left by *n* places

Implementation

```
1 a = a << zext16(n)
```

4.6.8 shri - logical shift right immediate

os	op	ds	vl	ff
0x00	0x17	a	n	0x01

Shifts the value of a in-place right by n places

Implementation

```
1 a = a >> zext16(n)
```

4.6.9 shri - arithmetic (signed) shift right immediate

os	op	ds	vl	ff
0x00	0x17	a	n	0x02

Shifts the value of a in-place right by n places, sign-replicating the result

Implementation

```
1 a = a >> (a >> zext16(n)) | (a >> zext16(n) >> (31 -
    zext16(n)) << 31)
```

4.7 Interrupt control

4.7.1 int - raise software interrupt

os	op	ds	ra	rb	ff
0x00	0x17	id	–	–	–

Raises a software interrupt

4.7.2 inti - raise software interrupt immediate

os	op	ds	vl	ff
0x00	0x18	id	–	–

Raises a software interrupt

4.7.3 intm - mask interrupt

os	op	ds	ra	rb	ff
0x00	0x19	id	enable	–	–

If *enable* is non-zero masks an interrupt, telling the CPU not to handle it through a Interrupt Routine Handler. Otherwise, (re-)enables it.

Implementation

```
1 if enable {
2     rrap[id * 0xA] = rrap[id * 0xA] | 1
3 } else {
4     rrap[id * 0xA] = rrap[id * 0xA] & ~1
5 }
```